From Monoliths to Pipelines: Practical Deployment of Multi-Billion Parameter Diffusion Models on Edge Devices

Chiheb Boussema and Jeff Burke

Center for Research, Engineering, Media and Performance (REMAP),
University of California Los Angeles

Revised March 26, 2026

1 Introduction

The recent advancements in generative AI are upending the entertainment industry and are opening up novel avenues for production, consumption, interaction, and ultimately novel experiences (Winick et al., 2025). While generative AI deployment usually implies cloud infrastructure (Boussema et al., 2025), cost, scale, security, privacy, coordination, and reliable internet access all become potential impediments to scalability. To mitigate this, we set out to deploy generative models in edge devices (e.g., smartphones), allowing users to bring their own devices to participate. This has the potential to massively expand the scale of these novel experiences, and allow both on-site (e.g., in a concert) and remote (e.g., from your living room) participation, not to mention AI-assisted day-to-day personal activities (e.g., smart glasses-smartphone workflows). High quality models enabling production-level generations however tend to be (i) large and operate on large tensors exceeding the limited memory available on edge devices, (ii) operate on and produce tensors of dynamic shapes, (iii) have operations that are trivially-compatible with pytorch/tensorflow and PC GPU (Nvidia, AMD, Apple Silicon) but are not exportable otherwise, (iv) implicitly rely on massive GPU parallelization hiding inefficient coding, and (v) involve denoising loops (for diffusion models) that would produce massively large graphs if exported as a monolith and likely resulting in PC crash during export.

All of the above makes (usable) generative models tricky to deploy to edge devices as they are unsuitable for a monolithic export. This manuscript aims to provide some of the practical strategies we employed to successfully deploy Stable Diffusion XL (SDXL, ~3.5B params.) (Podell et al., 2023) for image generation, and Stable Point-Aware Reconstruction of 3D Objects (SPAR3D, 3-4B params.) (Huang et al., 2025).

While each model export comes with its own unique set of challenges, the insights covered through these two examples are largely applicable to other diffusion models and provide useful guidance for future similar endeavors.

We end the manuscript with a brief discussion of our custom edge inference engine, built as a lightweight wrapper around Qualcomm's SNPE with the core principles of API simplifica-

tion, efficient memory management, and facilitation of edge-native hybrid neural-symbolic (diffusion) pipelines coding.

Our code is available at `https://github.com/remap/SNPEchainingDemo.git`.

2 Strategies for Export & Deployment

2.1 Model Decomposition & Hybrid Neural-Symbolic Pipelines

Unlike smaller, end-to-end networks common for example in computer vision tasks (e.g., YOLO) or even early small diffusion models, moderate to high-quality generative models are hardly exportable as a single monolith for the reasons outlined above. Additionally, these generative pipelines are often a combination of neural inference and more or less sophisticated code (e.g., geometric processing) that handles various intermediate steps. While it is at times possible to export this code as an ONNX graph, it is often more judicious to re-write this code in C++. This means that the diffusion pipeline is to be reproduced as a custom edge-optimized C++ code weaving neural inference on NPU and CPU/GPU processing in what can be termed a hybrid neural-symbolic edge pipeline.

Several factors influence how a generative model/pipeline should be decomposed:

- Function: Different modules or model fragments have distinct functional roles and provide natural splitting or chunking points. For example, the text and vision encoders and the VAE can be extracted separately from a UNet. Moreover, within a UNet, downsampling blocks can be exported separately from upsampling blocks.
- Size: Individual modules should be exportable as ONNX (ideally as a single file for ease of downstream processing, which limits exports to ~2GBs). Additionally, whether they are quantized and compiled locally or through cloud APIs (e.g., Qualcomm AI Hub), the size of the network influences the time and ability to compile. Networks that are too large risk running into time limits for cloud-based work (e.g., 1h limits), or RAM limits for local work. Finally, as further explained below, the size is also important when deployed on edge devices since large networks run the risk of crashing the device.
- Unexportable operations: Some ops cannot be exported from Pytorch to ONNX/DLC formats. These for example include random number generation. Others can be exportable but become problematic, for example exponential activation functions that can overflow on NPU. When these ops cannot be modified into equivalent but exportable ops, they provide splitting points.
- Iterative loops: Iterative (e.g., for or while) loops that iterate over large network inference provide a natural splitting point because export unrolls these loops and results in unruly graphs that hogs all system RAM. The iterated network should be isolated for export, and the loop coded in C++.
- Sensitivity to quantization: Some modules or networks are particularly sensitive to quantization. While quantization schemes often allow you to keep certain layers unquantized, it can also make sense to extract these as individual networks. For example, the time embeddings and conv-in and conv-out of a UNet.

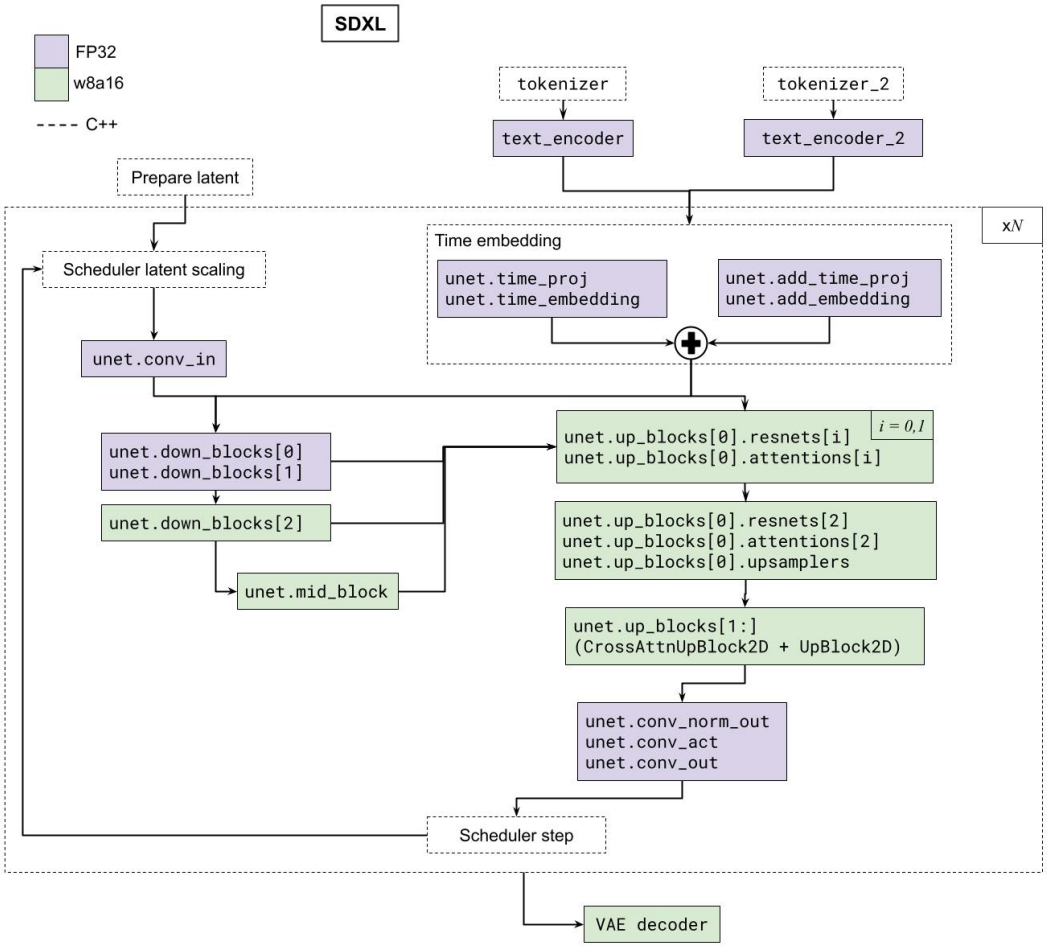

Figure 1: Decomposition of SDXL. Dashed lines indicate native C++ coding, as opposed to direct network export.

Memory management: beyond exportability, decomposition is also essential in order to run larger models on edge: model fragments can be loaded and unloaded sequentially to RAM, allowing a large model that cannot fit as a monolith to still run on edge. This is similar in spirit to CPU model offloading on PC: modern libraries allow loading parameters to GPU dynamically depending on operations, while the remaining parameters sit on CPU, thus allowing modest hardware to run larger models.

In our implementation, individual networks or modules were in general kept less than 2GB.

SDXL decomposition Our custom decomposition of SDXL is shown in Fig. 1. Sensitive networks have been kept in FP32, while others were quantized. The denoising loop is carried out natively in C++.

SPAR3D decomposition Our custom decomposition of SPAR3D is shown in Fig. 2. We will briefly mention here some of the custom adaptations. We refer the reader to Sec. 2.3 for more details.

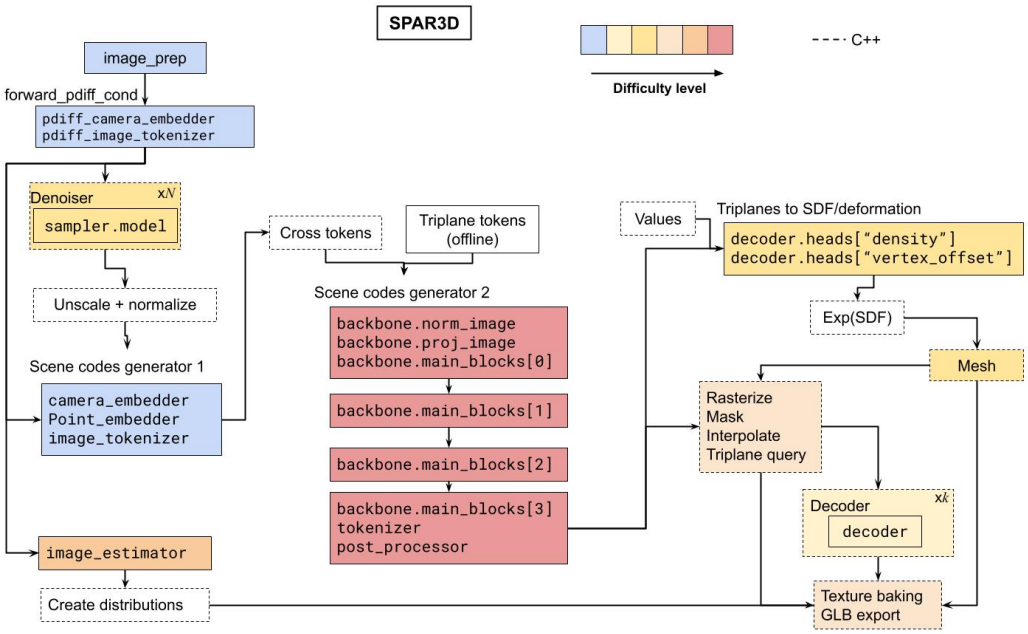

Figure 2: Decomposition of SPAR3D. Dashed lines indicate native C++ coding, as opposed to direct network export.

The denoiser network is exported with a custom masked multi-head attention to account for custom padding, and CFG generation is split into two passes rather than a single 2-batch pass. The denoising loop is carried out natively in C++. The image estimator involves a large ViT with 14x14 kernel. This was replaced by a depthwise convolution, with appropriate weight reconfiguration.

The transformer-based network forming the "Scene codes generator 2" has been split into chunks of manageable number of layers, with custom cross-attention and MLP chunking.

The decoder was exported expecting a fixed input size and outputting tensors of the same size D. If the actual inputs have size smaller than D, then we do zero-padding. If input size is greater than D, we loop over the decoder, feeding it chunks of size D. This is possible because the operations are point-wise (i.e., row-independent).

2.1.1 Code Optimization

Deploying the pipeline to the edge required moving beyond just optimizing the neural network; the surrounding pre-processing and post-processing logic had to be at times completely overhauled. Some of the original pipeline relied on Python and PyTorch operations that were either fundamentally unexportable to edge neural processing units (NPUs) or were written with the assumption of a high-bandwidth desktop GPU. Translating these components line-by-line into C++ was insufficient; the algorithms themselves had to be redesigned for a mobile CPU architecture.

Algorithmic Redesign Over Direct Translation A major challenge in edge deployment is that desktop Python code often relies on the sheer compute power of a discrete GPU to hide algorithmic inefficiencies.

An example of this is SPAR3D's rasterizer. The original codebase implemented mesh rasterization using a pixel-centric Bounding Volume Hierarchy (BVH) approach. It iterated over every pixel in the output image and performed a computationally expensive tree-traversal intersection test. While tolerable on high-end hardware, this caused severe branch mispredictions and cache thrashing on a mobile CPU. We completely redesigned this module into a Tile-Based Barycentric Rasterizer. By pre-projecting triangles into 2D space, binning them into 16x16 spatial tiles, and evaluating pixels using optimized edge functions, we localized memory access to the CPU's L1 cache and eliminated the massive overhead of per-pixel tree traversal. Furthermore, assigning tiles to specific threads removed the need for atomic locks, drastically increasing multi-threading efficiency.

Cache-Friendly Memory Paradigms vs. GPU Tensor Paradigms A critical architectural shift involved replacing GPU-oriented tensor operations with cache-friendly CPU memory paradigms. Desktop GPUs possess immense memory bandwidth and easily hide the overhead of massive memory allocations, making operations like unfolding large tensors for parallel convolutions highly efficient. Conversely, mobile CPUs are severely memory-bound; their performance relies heavily on keeping data tightly localized within ultra-fast, but extremely limited, L1 and L2 caches. For example, during the pipeline's texture baking phase, the original Python code utilized a tensor unfold mechanism to bleed texture edges. Executing this direct translation on a mobile CPU required allocating hundreds of megabytes of temporary arrays per iteration, which immediately thrashed the CPU's cache, choked the system's memory bandwidth, and inflated the execution time to over 30 seconds. We resolved this by discarding the tensor-based convolution entirely in favor of a direct memory-access algorithm in C++. By utilizing raw pointers and executing a localized 8-way neighbor lookup, we kept the active image chunks perfectly contained within the L1 cache. This cache-conscious redesign eliminated all intermediate massive allocations and slashed the processing time from 30 seconds down to roughly 2 seconds.

2.2 Distillation & Quantization

2.2.1 Distillation

Distilling diffusion models into one- or few-step generation is a typical method for reducing latency and approaching real-time generation. On the flip side however, aggressive distillation is generally associated with lower quality generation. Depending on the application, perceived quality of the generated media may be more or less important. In our case, it was of relatively high importance.

While we did not distill models ourselves, we chose to work with ByteDance's 8-step distillation of SDXL (Lin et al., 2024). This provided a good balance of a speedy generation while retaining adequate image quality.

2.2.2 Quantization

Quantization is another important practical step towards edge deployment: it both lowers memory requirements and speeds up inference.

As diffusion models rely on iterative denoising loops, small quantization-introduced noise can be highly amplified and lead to unacceptable results.

In our case, we successfully quantized most of SDXL's UNet (most of the downblocks, the middle block, and the upblocks) and the final VAE. We found that quantizing the weights to INT8 and the activations to INT16 (w8a16) led to nearly 50% reduction in inference time while keeping quality virtually identical. The text encoders and smaller networks at the start and tail of the UNet were kept at FP32. The text encoders in particular are highly sensitive to quantization.

As for SPAR3D, quantization efforts were unfortunately unsuccessful. While inference time was improved substantially, the final results were of an unacceptably lower quality.

Typical quantization procedure Deploying a quantized network to edge devices follows a tedious but simple recipe.
After a model decomposition has been decided,

- Run a forward pass of the model and gather for each exported module its input and output tensors. Do this with various initial inputs (e.g., text prompts) and seeds.
- For each module,
  1. Export to ONNX (`torch.onnx.export`).
  2. Simplify the ONNX graph with `onnx-simplifer` .
  3. Quantize the simplified onnx with `aimet-onnx` (see documentation on their website). This will require coding a simple data loader, a forward pass function, and optionally an evaluation function (SNR is recommended as an evaluation metric). Compute the encodings then export the AIMET model.
  4. Compile and quantize the AIMET model, either locally through the QAIRT SDK, or through the Qualcomm AI Hub (QAI). The latter is simpler in our opinion, and can be done by simply calling:

     ```
     import qai_hub as hub
     job = hub.submit_compile_job(
         model='model_name.aimet',
         name='model_name',
         device=hub.Device("Snapdragon 8 Elite QRD"),
         options="--target_runtime qnn_dlc
             --quantize_full_type w8a16 --qairt_version
             2.43",
         calibration_data=calibration_dataset
         )
     ```

     If quantizing, it is preferable to upload a calibration dataset rather than let the SDK make a random one. This is to ensure data values remain within expected range (for example the timestep for the denoiser). Otherwise, one may get error messages that are not immediately obvious.
  5. After the module has been compiled to DLC, inspect it locally with the QAIRT SDK's `snpe-dlc-info -i network_name.dlc` . Note expected input and output names and check shapes (sometimes the graph may cut a final transpose).

6. Prepare the graph for your specific edge device with `snpe-dlc-graph-prepare`.

2.3 Architectural Adaptations

In addition to model decomposition and pipeline re-writing, have to make changes. Can be simplified forward if clean extraction, or a simplified wrapper around a block of inference+processing (and here need to decide what the inputs and outputs are), or more involved like changing the architecture, the nn modules, their forward function, the tensors.

In addition to model decomposition and pipeline re-writing, we found it necessary in some situations to modify the model in various ways. Common modifications include simplification or modification of the forward method of some network (`nn.Module`) without modifying its inputs or outputs, and modifying the forward method in a way that changes the inputs or outputs. For example, output a list of tensors instead of a dictionary, or accepting a random tensor as input instead of creating it inline. More generally, for moderately large tensors created inline during function calls yet have fixed values, we recommend either generating them offline, making them inputs to the corresponding networks, and importing them into the native app.

Some unexportable operations may involve alternative coding, or premature trunking of the network operations (see the network decomposition section) so that the unexportable operations can be naitvely coded in C++ on the edge device.

As an example, we found that exponential activation functions can be problematic. This was the case when inferring the signed distance function of the 3D object in SPAR3D. Although in principle exportable, deployment on NPU resulted in Infinite values. Moving the exponential activation to native CPU with input clamping (to ±80) resolved the issue without noticeable added latency.

Another example is the inference of certain object properties (e.g., roughness and metallic). The network predicts the parameters of a Beta distribution, from which the actual properties are derived. As distributions cannot be exported, we limited the output of the network to the distribution properties, then re-created the distribution and sampling process natively on edge.

Changing the forward method may at times require modifying the entire `nn.Module` class. As diffusion model modules are often made of a recursive imbrication of modules, changes to one level may require adapting the upper levels. For example, in SPAR3D, changing the cross attention module implies changing the `FusedBlock` and `BasicBlock` modules, which in turn implies changing the `TwoStreamBlock` module.

Beyond this, more involved architectural changes were necessary for successful deployment, in particular for SPAR3D.

2.3.1 Adapting to Edge Memory Constraints: Tiling, Alignment, and Compute Restructuring

Preliminary considerations PC and mobile hardware architectures are different. On a PC, CPU has its own system RAM (DDR5), while the GPU has its own dedicated VRAM (GDDR6), with a massive memory bus that allows blasting through the matrix math directly

inside the VRAM. Mobile chips on the other hand use a Unified Memory Architecture (UMA), whereby CPU, GPU, and NPU all share the exact same physical LPDDR memory chips. Moving data physically from the motherboard's RAM to the processor costs orders of magnitude more energy than doing the math, and while modern mobile RAM is fast, it is several folds slower than modern GPUs.

To mitigate this, mobile devices use a Tightly Coupled Memory (TCM), a sort of ultra-fast SRAM of a few MBs (e.g., 8MB) physically baked directly onto the NPU silicon, right next to the math units that acts as the efficient workbench. The lifecycle of a tensor operation on the edge then looks something like this:

- Tiled Fetch: Compiler schedules movement of input/weight tiles from RAM to on-chip TCM/SRAM, minimizing bandwidth and maximizing reuse.
- Local Compute: Compute units operate on SRAM-resident data with extremely high local bandwidth and much lower energy than RAM access.
- Data Reuse + Fusion: Intermediate results are kept on-chip as long as possible. Operator fusion and dataflow strategies (e.g., output-stationary) reduce memory traffic.
- Spill / Write-back: Only when necessary (once the TCM workbench gets too crowded, or the chain of operations is finished), results are written back to RAM.

A key aspect that concerns us is the tiled fetching. If this fails, then the whole edge computations become highly inefficient. Mobile DSPs, such as the Snapdragon Hexagon, enforce proprietary memory formats (e.g., the Crouton format) to accelerate integer and floating-point matrix math. These hardware tensor cores physically require sequence lengths and channel dimensions to be strictly aligned to multiples of 8 or 32. If the compiler cannot figure out how to slice the matrices to fit onto its TCM workbench, it will give up on fusion and do the math by constantly swapping massive, unaligned temporary arrays back and forth to the System RAM "warehouse" (memory spill). This causes the RAM to fill up with junk, and can even cause the exporting PC to crash or stall trying to map it out.

We will in the following discuss strategies we employed to mitigate these edge-specific constraints.

Padding Unaligned sequences forced the offline compiler to inject thousands of hidden memory-reformatting nodes, resulting in massive RAM usage, compute overhead, and high memory spill. By artificially padding tensors to achieve a final sequence length that is a multiple of 8, we allow the compiler to route the tensors natively through the hardware, eliminating formatting penalties.

In addition to padding for alignment purposes, some modules can output or accept tensors of variable size (e.g., the number of valid vertices for a mesh). Dynamic shapes are incompatible with edge deployment. In such cases, we resorted to zero-padding up to a fixed size. However, it is possible that some cases exceed this fixed size. For these cases, we leveraged the point-wise independence of the downstream networks. Because network layers operate on each vertex independently without cross-row interaction, we could safely slice oversized vertex arrays into fixed-size chunks, process them sequentially through the NPU, and

concatenate the outputs on the CPU. This bypassed the NPU's fixed-shape limitations without altering the mathematical integrity of the network. Note that point-wise (or row-wise) independence is not guaranteed and depends on the network architecture. It is therefore important to verify beforehand.

Working Memory Limits While the QAIRT compiler features automatic spatial tiling for aligned tensors, it cannot automatically optimize the network's temporal memory footprint (the High Water Mark). Operations requiring global context (like Softmax) or massive feature expansion (like MLPs) bypass the automatic tiler and exceed the TCM capacity. Therefore, we manually restructured the compute graph to enforce chunking at the architectural level, mathematically guaranteeing (as much as possible) that the peak memory footprint of any single operation never breached the hardware limit.

An example of where memory limitations can intervene is for classifier-free guidance (CFG) during generation. CFG requires inference on a conditioned and on an unconditioned input. These two are generally passed as a single input with batch size B = 2. By breaking conditional and unconditional passes into separate executions (CFG splitting), we alleviate memory footprint. We did this for SPAR3D's denoiser module.

For similar reasons, we found these additional strategies to be important:

Restructuring the Attention Mechanism We implemented several structural changes to the attention blocks to make them DSP-friendly:

- Attention chunking: To prevent the massive N x N attention probability matrix from overflowing the TCM, queries, keys, and values were sliced into processable chunks at two steps of the cross attention:

  (i) when computing the q,k,v values from the inputs; this is also where we perform channel permutation/transpose for the multihead attention. We further split q-related operations from kv-related operations, since their sizes are different for cross-attention (and hence handle different operational batch sizes).

  (ii) when computing the scaled dot product attention. The scaled dot product attention is particularly tricky as the attention score requires the full KV tensors. When the input tensors for the keys and values are large (e.g., 27'648 x 1024), this can hog large memory and force the query batch to be very small. Splitting the attention computation with appropriate batches of queries (i.e., q-wise chunking) allowed the hardware to execute the attention mechanism sequentially within the physical limits of the SRAM.

- Minimizing Permutations: Operations like transpose and permute are effectively "free" on a GPU but are highly expensive on an NPU, as they destroy the linear memory contiguity required by the DMA (Direct Memory Access) engine. Wherever possible we refactored the attention code to minimize memory-shuffling operations before the Softmax step.

- Masked Attention: As noted before, it is at times necessary to artificially pad tensors for TCM tiling alignment. When these tensors need to pass through an attention

block, it is necessary to augment the function with a mask for the query or keys/values (value of the mask at padded dimensions should be a large negative number). The original model code may not natively support it, requiring re-coding of the attention class, its forward method, and upper level torch modules as fit.

MLP Feature Expansion Multilayer Perceptrons (MLPs) in modern transformer blocks typically expand feature dimensions by a factor of 4x or 8x before projecting back to the original dimension. While trivial on a GPU, this sudden 4x explosion in tensor size mid-layer acts as a severe bottleneck on the edge, instantly exceeding the TCM limit and triggering a RAM spill. To mitigate this, the MLPs were explicitly profiled, and processing was batched or sliced to ensure the expanded hidden states remained within the bounds of the fast working memory.

Examples For a particular module of the transformer-based scene encoder, attention q-chunking, minimizing permutations, and chunking the MLP stage reduced memory spill from 7.5GB to 0.5GB.

For SPAR3D's denoiser, simply padding input tokens to a multiple of 8 (and making sure to change the attention to masked attention) and performing a CFG splitting decreased memory spill from 50GB to 7GB, and vastly reduced compiler time.

From Large Convolutions to Space-to-Depth The standard Vision Transformer (ViT) architecture extracts image patches using a Convolutional layer with a large kernel and stride (e.g., 14x14). Mobile NPUs, however, feature hardware accelerators highly optimized for 1x1 or 3x3 convolutions; large kernels stall the compute units and exhibit terrible DSP utilization and can even resist compilation.

- Pixel Unshuffle: We replaced the 14x14 patch convolution entirely. Instead, we utilized a `pixel_unshuffle` (Space-to-Depth) operation, which natively and efficiently maps spatial dimensions into the channel dimension.
- Weight Reconfiguration: Following the Space-to-Depth operation, we applied a highly optimized 1x1 Depthwise Convolution. Because the mathematical operations changed, we had to write custom offline scripts to permanently permute and reconfigure the pre-trained weights, ensuring the numerical output remained identical to the original ViT while executing at a fraction of the hardware cost.

3 Architectural Framework: A Zero-Copy, Neuro-Symbolic Edge Engine

Deploying complex generative models on mobile hardware requires more than just compiling neural networks; it requires an orchestration layer capable of managing extreme memory constraints while seamlessly interleaving deep learning models with traditional native algorithms. Qualcomm's native SNPE C++ API, while powerful, is quite verbose, requiring extensive boilerplate code per model to manage runtimes, user buffers, and tensor mappings, and is not beginner-friendly.

To overcome this, we designed and implemented a custom, lightweight inference engine acting as a wrapper around SNPE. The philosophy of this framework was built on three core pillars: API simplification, zero-copy memory management, and the facilitation of hybrid neuro-symbolic pipelines.

3.1 Configuration-Driven API Simplification

To remove the friction of manually wiring neural networks together, the framework abstracts model initialization and execution into a configuration-driven API. Rather than hardcoding the instantiation of the denoisers, VAEs, or projection matrices, the system relies on `PipelineCfg` structures (simple JSON-formatted configurations files).

By simply passing a configuration file to the `buildArbitraryChainFromConfig` helper, the framework automatically parses the required models, determines their input/output shapes, selects the optimal hardware runtime (DSP, GPU, or CPU), and wires the execution graph. Once built, executing an entire chain of models—regardless of complexity—is reduced to a single, high-level `runGraph()` call. This dramatically reduced boilerplate code and accelerated the prototyping of complex multi-stage pipelines like SDXL and SPAR3D.

3.2 Centralized Memory Management and Zero-Copy Routing

On unified memory architectures, redundant memory copying between model executions is fatal to both latency and battery life. To solve this, we implemented a centralized memory arena called the `TensorWorkspace`.

The `TensorWorkspace` owns all raw byte allocations for the entire pipeline. Instead of allowing SNPE to internally allocate and manage tensor memory, we utilized SNPE's `UserBufferMap` interface (via `CreateUserBuffer` and our custom `ModelSession`) to force the NPU to read from and write directly to our pre-allocated workspace.

Crucially, the workspace supports strict aliasing. If "Model A" outputs a tensor that "Model B" consumes, `GraphRunner` binds both models to the exact same memory address in the `TensorWorkspace`. Zero bytes are copied during the handoff. Furthermore, the orchestrator exposes aggressive lifecycle management functions (e.g., `clear_everything_in_session()`). Because memory is strictly centralized, we can dynamically tear down heavy model sessions as soon as they finish executing, freeing system RAM for downstream steps.

3.3 Facilitating Hybrid Neuro-Symbolic Pipelines

Modern AI models are rarely "pure" deep learning; they usually require blending neural network inference with traditional, algorithmic code (symbolic execution). By abstracting the deep learning execution into `GraphRunner` nodes and keeping all data centralized in the `TensorWorkspace` as raw C++ pointers, the framework acts as a perfect bridge for neuro-symbolic pipelines.

This is demonstrated in the `Spar3DPipeline`. The pipeline executes a chain of deep learning models on the NPU to estimate, for instance, latent scene codes, images, and triplanes. Because the output tensors are natively exposed in the `TensorWorkspace`, the

pipeline can seamlessly hand those raw float pointers directly over to purely symbolic, native C++ algorithms. For example, the NPU outputs are immediately consumed by our custom CPU-optimized `MarchingTetrahedraHelper` to extract the 3D geometry, followed by `MeshCPP` and `baker_cpu` to finalize the rasterization and texture bakin

By unifying the neural network execution and the symbolic algorithmic processing under one continuous, zero-copy memory space, the framework achieves maximum hardware utilization without the serialization penalties of crossing runtime boundaries, unlocking complex, multi-stage generative AI on the edge.

Acknowledgments

We appreciate work by the UCLA REMAP team to motivate and test the Snapdragon ports, including: Naisha Agarwal, Bianca Bishop, Laine Matkin, Aidan Strong, Camilo Vargas, and Mira Winick. This work was supported by Qualcomm. We appreciate the technical input and references provided by Qualcomm employees throughout the project.

References

Boussema, C., Agarwal, N., Vargas, C., Wilson, M., Doolan, A., Browning, A., Winick, M., and Burke, J. (2025). Xanadu: Generative media pipelines for immersive participatory theater. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 21, pages 389–393.

Huang, Z., Boss, M., Vasishta, A., Rehg, J. M., and Jampani, V. (2025). Spar3d: Stable point-aware reconstruction of 3d objects from single images. *arXiv preprint arXiv:2501.04689*.

Lin, S., Wang, A., and Yang, X. (2024). Sdxl-lightning: Progressive adversarial diffusion distillation. *arXiv preprint arXiv:2402.13929*.

Podell, D., English, Z., Lacey, K., Blattmann, A., Dockhorn, T., Müller, J., Penna, J., and Rombach, R. (2023). Sdxl: Improving latent diffusion models for high-resolution image synthesis. *arXiv preprint arXiv:2307.01952*.

Winick, M., Agarwal, N., Boussema, C., Lee, I., Vargas, C., and Burke, J. (2025). Ai as intermediary in modern-day ritual: An immersive, interactive production of the roller disco musical xanadu at ucla. *arXiv preprint arXiv:2511.06195*.